

COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS, PHYSICS AND INFORMATICS

HISTORY PRESERVING FILESYSTEM FOR LINUX
BACHELOR THESIS

2026
DMYTRO PETRENKO

COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS, PHYSICS AND INFORMATICS

HISTORY PRESERVING FILESYSTEM FOR LINUX

BACHELOR THESIS

Study Programme: Computer Science
Field of Study: Computer Science
Department: Department of Computer Science
Supervisor: RNDr. Jaroslav Janáček, PhD.

Bratislava, 2026
Dmytro Petrenko



Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Dmytro Petrenko
Študijný program: informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: anglický
Sekundárny jazyk: slovenský

Názov: History Preserving Filesystem for Linux
Históriu zachovávajúci súborový systém pre Linux

Anotácia: Častým problémom v súčasných operačných systémoch je neúmyselné alebo úmyselné vymazanie alebo prepísanie súboru s dôležitými údajmi. Cieľom bakalárskej práce je navrhnúť a implementovať súborový systém pomocou FUSE, ktorý umožní efektívne udržiavať históriu zmenených a vymazaných súborov a umožní získať prístup k predchádzajúcemu obsahu.

Cieľ: navrhnúť efektívny spôsob ukladania histórie obsahu súborov
implementovať pomocou FUSE vrstvu súborového systému zachovávajúcu históriu
navrhnúť a implementovať nástroje na prístup k historickému obsahu
otestovať vytvorené dielo

Vedúci: RNDr. Jaroslav Janáček, PhD.
Katedra: FMFI.KI - Katedra informatiky
Vedúci katedry: prof. RNDr. Martin Škoviera, PhD.
Dátum zadania: 15.10.2025

Dátum schválenia: 27.10.2025

doc. RNDr. Dana Pardubská, CSc.
garant študijného programu

.....
študent

.....
vedúci práce



THESIS ASSIGNMENT

Name and Surname: Dmytro Petrenko
Study programme: Computer Science (Single degree study, bachelor I. deg., full time form)
Field of Study: Computer Science
Type of Thesis: Bachelor's thesis
Language of Thesis: English
Secondary language: Slovak

Title: History Preserving Filesystem for Linux

Annotation: Accidental or intentional deletion or overwriting of files containing important data is a common problem in current operating systems. The goal of this bachelor's thesis is to design and implement a filesystem layer using FUSE that will effectively preserve history of changed and deleted files, and that will enable access to the previous content of the files.

Aim: design an effective way to store history of files' content
implement, using FUSE, a filesystem layer preserving history
design and implement tools to access the historical content
test the created work

Supervisor: RNDr. Jaroslav Janáček, PhD.
Department: FMFI.KI - Department of Computer Science
Head of department: prof. RNDr. Martin Škoviera, PhD.

Assigned: 15.10.2025

Approved: 27.10.2025 doc. RNDr. Dana Pardubská, CSc.
Guarantor of Study Programme

Student

Supervisor

Declaration: I hereby declare that I have independently written this entire bachelor thesis entitled “History Preserving Filesystem for Linux”, including all its figures and appendices, using the literature listed in the attached bibliography.

In preparation of the thesis I have also used the following artificial intelligence tools: ChatGPT (free version) [8] for the following purposes: language proofreading and grammar correction to ensure the correctness and clarity of the written text, and assistance in searching for relevant literature sources. The tool was used only for identifying and locating potential sources and was not used as a source of information itself. I have used artificial intelligence tools in accordance with relevant legal regulations, academic rights and freedoms, ethical and moral principles while simultaneously maintaining academic integrity. I am aware that I am fully responsible for the accuracy of the final text.

Acknowledgments: I would like to thank all lecturers involved in the *Bakalársky seminár* course for their guidance and valuable feedback throughout the writing of this thesis. Their advice and discussions helped me gain a better understanding of my work and plan the individual stages of the project.

Special thanks go to my supervisor RNDr. Jaroslav Janáček, PhD., for the opportunity to develop this project under his supervision. He became a great inspiration to me throughout my studies and showed me that becoming a skilled specialist requires continuous hard work and dedication, which motivated me during the entire development of this project. I am also very grateful for the many discussions and meetings we had, as they greatly helped me understand the problem and improve the quality of this thesis and the implemented filesystem.

Abstract

This bachelor's thesis focuses on the design and implementation of a Linux file system that preserves history of the files. This file system addresses the issues when working with sensitive files such as accidental or intentional deletion or overwriting of files. The goal of this thesis is to implement a file system that makes loss of data recoverable and gives opportunity to go back to older versions of the files. The thesis implements the system and measures memory usage and access time to different versions of the file. As a result of this thesis, the file system may be mounted and used to work with sensitive files.

Keywords: file system, file versioning, sensitive files, performance

Contents

Introduction	1
1 Related Work	3
2 Architecture and Design	7
3 Implementation	11
3.1 Versioning Design	13
3.2 Optimization and details	15
3.3 Version Creation and Internal File Updates	20
3.4 Methods of Accessing File Versions	21
4 Testing	25
4.1 Testing write	25
4.2 Testing read	26
5 Alternative approach	29
5.1 Implementation details	29
5.2 Testing the new approach	31
Future Work	33
Conclusion	35
References	37
User Manual	39

List of Figures

2.1	A flow-chart diagram showing how FUSE works	8
3.1	Two-layered inode structure	12
3.2	Structure of a versioned file	14
3.3	Optimization figure	16
3.4	Special pointer	16
3.5	Example of a version file.	19
3.6	Mounted vs. umounted directory	19

List of Tables

4.1	Write performance comparison and slowdown relative to passthrough .	26
4.2	Read performance of different file versions	27
5.1	Read performance of different file versions	31

Introduction

Modern computer systems rely heavily on file systems to store, organize, and access data. As the amount of stored information continues to grow, protecting files against accidental modification, corruption, or malicious activity becomes increasingly important. Traditional file systems are primarily designed to provide efficient storage and fast access to data, but they often do not offer sufficient mechanisms for recovering previous states of files after unwanted changes occur. As a result, additional systems such as backups, snapshots, or version control solutions are commonly used to improve reliability and recoverability.

One possible approach to solving this problem is the use of versioned file systems. A versioned file system preserves older states of files and allows the user to restore previous versions when necessary. Such systems can be useful in many situations, including accidental deletion, incorrect modifications, software failures, or attacks that corrupt user data. Maintaining file history directly at the file system level also provides transparency to applications, since versioning is handled independently of user software.

This thesis focuses on the implementation of a history-preserving file system using FUSE (Filesystem in Userspace). FUSE makes it possible to implement custom file system logic in user space without modifying the operating system kernel. This significantly simplifies development and experimentation while still allowing complete control over file operations such as reading, writing, opening, and closing files.

The implemented system stores multiple versions of files and allows recovery of previous states after modifications occur. The thesis primarily considers the system as a recovery mechanism, where the most important requirement is efficient access to the current version of a file during normal operation, while older versions are preserved for exceptional situations. Because of this, the design decisions throughout the thesis focus on balancing versioning functionality with acceptable performance overhead.

Several implementation approaches are examined in this work. The first implementation prioritizes fast access to the current version of a file while allowing reconstruction of older versions from stored metadata and overwritten blocks. This approach introduces slower access to historical versions, but minimizes overhead during standard usage where the current version is accessed most frequently.

An alternative implementation is also proposed and evaluated. In this approach,

all versions are stored in a way that allows them to be accessed in approximately the same amount of time. This simplifies version reconstruction and improves access to historical versions, but introduces additional overhead when reading the current version and during write operations. The thesis compares both approaches and analyses the trade-offs between them.

An important part of this work is performance evaluation. The implemented file systems are tested and compared against a traditional Linux file system and a basic FUSE passthrough implementation. The tests focus on both write and read operations in order to evaluate the overhead introduced by version management and the impact of different design decisions.

The goal of this thesis is not only to implement a functional versioned file system, but also to analyse the consequences of different approaches to storing and reconstructing file versions. Particular attention is given to the relationship between performance, storage overhead, and accessibility of historical data.

The thesis is structured as follows. First, the theoretical background related to file systems, FUSE, and versioning techniques is introduced. Afterwards, the design and implementation of the proposed file systems are described in detail. The following chapters focus on testing, evaluation, and comparison of the individual approaches. Finally, the obtained results are discussed and possible future improvements are presented.

Chapter 1

Related Work

In this chapter, we discuss various existing file systems and their approaches to data recovery and file versioning. For each system, we highlight the primary goal and the mechanisms it provides for preserving or recovering data.

Traditional Linux file systems. The primary goal of traditional file systems such as ext4 [5] and XFS [7] is to be efficient, reliable, consistent and maintain data integrity. They usually are not designed for preserving file history. Traditional file systems focus on performance and integrity but do not provide per-file version history. This motivates the design of a file system that preserves file history while remaining efficient. The proposed system keeps the standard file operations provided by conventional Linux file systems, while extending functionality to maintain history of versions of the file.

Btrfs File System. Btrfs is a modern Linux file system that introduces several advanced features beyond traditional file systems [1]. Its copy-on-write (COW) mechanism ensures that existing data blocks are never overwritten. Reads access the original blocks, while writes create new copies for modifications. Metadata is then updated to reference the new blocks, allowing snapshots to remain consistent without duplicating unchanged data. Snapshots are read-only or writable images of the file system at a specific moment, enabling recovery of the entire system in case of corruption or accidental deletion. Btrfs also supports subvolumes, which are independently mountable file system trees, allowing logical separation of data within a single file system.

While these features make Btrfs highly reliable and flexible for system-wide data protection, its snapshot and versioning capabilities are mostly at the subvolume level, rather than individual files. In contrast, the proposed file system emphasizes per-file versioning, allowing users to recover specific previous versions efficiently, which is particularly useful for sensitive project data or frequently modified files.

ZFS File System. OpenZFS is an open-source storage platform that combines the functionality of a traditional file system with a volume manager [3]. It provides strong protection against data corruption through integrity checking of both data and metadata, along with continuous verification and automatic self-healing mechanisms. ZFS supports data redundancy using mirroring and RAID-Z configurations, enabling reliable storage even in the presence of hardware failures. The system is designed to scale to very large storage capacities and offers space-saving features such as transparent compression using algorithms including LZ4, GZIP, and ZSTD. Additionally, ZFS provides hardware-accelerated native encryption and supports efficient storage management through snapshots and copy-on-write clones. For data replication, ZFS enables efficient local and remote synchronization by transferring only modified data blocks. Snapshots can be created almost instantly, and they initially consume no additional disk space within the pool. However, as data within the active dataset changes, the snapshot consumes disk space by continuing to reference the old data, thus preventing the disk space from being freed [2].

Comparison and Relation to the Proposed File System. Btrfs and ZFS are comprehensive storage systems that provide a wide range of advanced features, including copy-on-write semantics, data and metadata checksumming, compression, RAID support, cloning, and volume management. Within these systems, snapshots represent an important but auxiliary mechanism that complements their primary goals of ensuring data integrity, scalability, and reliable storage management at the volume or dataset level. Their snapshot functionality is therefore designed to preserve the state of an entire file system or dataset rather than to maintain detailed histories of individual files.

In contrast, the file system proposed in this thesis has a significantly narrower and more specialized scope. Its primary objective is to preserve the history of individual files, enabling efficient recovery of previous versions without affecting unrelated data. While the granularity of versioning differs substantially from system-wide snapshots, the proposed system shares a key conceptual similarity with Btrfs and ZFS in that it is implemented efficiently: historical versions are stored using a design that avoids unnecessary data duplication and retains only modified data blocks.

Another distinguishing aspect of the proposed file system is its implementation using the FUSE(Filesystem in Userspace) framework [6], which allows the system to operate entirely in user space. This approach simplifies development, testing, and maintenance when compared to kernel-level file systems, while still supporting standard file operations. Unlike Btrfs and ZFS, the system does not aim to provide extensive storage management features or transactional guarantees; instead, it introduces targeted functionality focused on file-level versioning and fast access to historical data.

Furthermore, the internal structure of the proposed file system provides additional security benefits. The system is mounted using FUSE, where all file operations are performed through the mounted filesystem interface, while the underlying storage directory containing the actual data is not intended to be accessed directly by the user. This effectively makes the internal file structure inaccessible through standard file system operations, as all interactions must go through the implemented FUSE layer.

This design protects data not only from accidental deletion or overwriting but also from intentional tampering, such as unauthorized modification, encryption, or removal of files. The only way to bypass this mechanism is to unmount the file system, after which the underlying storage becomes directly accessible. However, this assumes that the attacker is not aware of the versioned file system. By combining controlled access with persistent file histories, the system ensures that previous versions remain recoverable even in adversarial scenarios.

In summary, Btrfs and ZFS are primarily designed to manage and protect entire storage systems and include snapshots as one of many advanced features. The proposed file system focuses specifically on per-file history, efficient storage of changes, and enhanced protection for sensitive data. This specialization makes it particularly suitable for use cases where histories of files and file-level security are of greater importance than full-scale storage management, such as personal work with documents or source code.

Chapter 2

Architecture and Design

The architecture of FUSE consists of two essential components: a kernel module and a user-space process. The kernel module operates within the operating system kernel and is responsible for intercepting filesystem-related system calls. It forwards these requests to the user-space component and subsequently returns the processed results back to the kernel.

The user-space component implements the actual filesystem logic. In this work, the libfuse library is utilized to facilitate communication between the user-space process and the kernel module. This library provides an interface through which filesystem operations are defined as callback functions. When a request is issued by the kernel module, it is delivered to the corresponding callback function implemented in user space.

Furthermore, the use of the libfuse library enables the implementation of custom filesystem operations. These operations are handled within the user-space process, where the developer can define specific behavior beyond standard filesystem functionality. Requests for such operations are transmitted from the kernel module to the user-space implementation via libfuse, processed accordingly, and the results are returned through the same communication channel.

In this thesis, FUSE is used to implement a history-preserving file system. The main idea is to use the directory where the filesystem is mounted as the storage location for the FUSE filesystem. This design has important security implications. Since all real data is managed through the FUSE layer, the underlying directory structure is not directly accessed during normal operation. As a result, an attacker interacting with the mounted filesystem can only perform operations through the implemented interface, which enforces versioning.

Another important aspect of this design is that the versioning mechanism is completely transparent to the user. Files are accessed and modified in the same way as in a traditional filesystem, while version management is handled internally by the FUSE

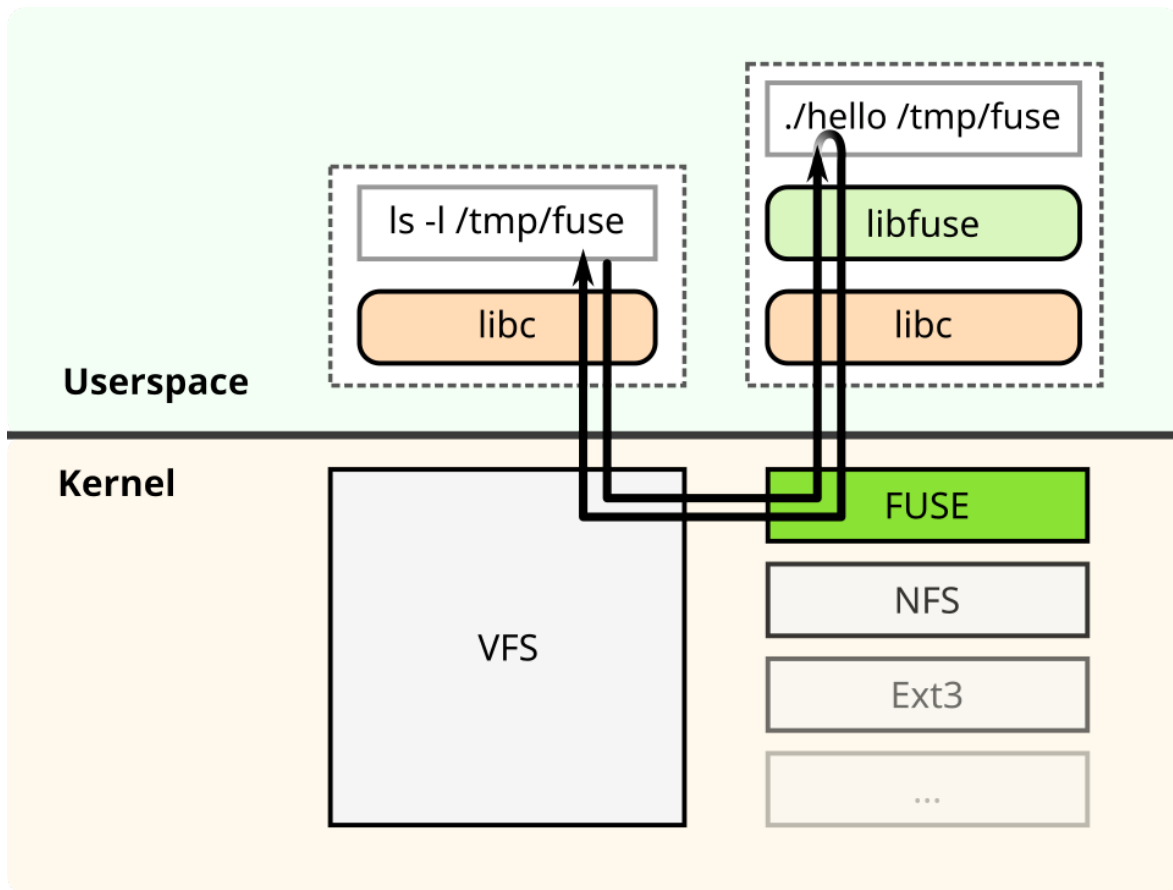


Figure 2.1: A flow-chart diagram showing how FUSE works: Request from userspace to list files (`ls -l /tmp/fuse`) gets redirected by the Kernel through VFS to FUSE. FUSE then executes the registered handler program (`./hello`) and passes it the request (`ls -l /tmp/fuse`). The handler program returns a response back to FUSE which is then redirected to the userspace program that originally made the request.

implementation. This allows file histories to be created automatically without requiring additional user interaction or external backup tools.

The only way to bypass the versioned FUSE filesystem is to unmount it, which would expose the underlying files directly. In this model, we assume that the attacker is not aware of the versioning mechanism and therefore operates only through the mounted interface. This assumption is reasonable in scenarios where the filesystem behaves like a normal storage location from the user's perspective, while internally enforcing additional protection mechanisms.

To illustrate this, consider an attack in which an attacker encrypts all files within the mounted filesystem. In such a scenario, the current file becomes corrupted, but the user is still able to restore previous versions of the file and recover the original data. Since modifications performed by the attacker are processed through the FUSE layer, the previous state of the file remains preserved by the versioning mechanism even after the malicious changes are applied.

Chapter 3

Implementation

In this chapter, the required properties of the file system are discussed, along with the approach used to achieve them.

The following list summarizes the key performance-related properties required of the proposed file system.

- Fast access to the current version of a file: The file system prioritizes efficient access to the most recent version of each file, as this is the version used during normal operation.
- Access to older file versions: Retrieval of historical versions is supported but may be slower, since these versions are accessed less frequently and may require additional lookup or reconstruction.

The initial approach followed the design principles of traditional file systems, extending them with support for file versioning. The design is based on the concept of inodes, extended to support file versioning. The structure consists of two layers of inodes. The higher-layer inode represents a file and contains the file metadata together with pointers to version structures. Each lower-layer inode represents a specific version of the file and contains pointers to the data blocks that correspond to that version. This two-layer inode structure is illustrated in Figure 3.1.

Now consider a file that is large enough to require the use of indirect and double-indirect blocks. In this case, creating a new version of the file would necessitate copying the entire inode structure, including all indirect and double-indirect blocks, for each version. While this design provides fast access to both the current and previous versions, the duplication of inode structures results in substantial space overhead. For large files, this overhead becomes significant, making the approach impractical for scenarios involving frequently modified, large files.

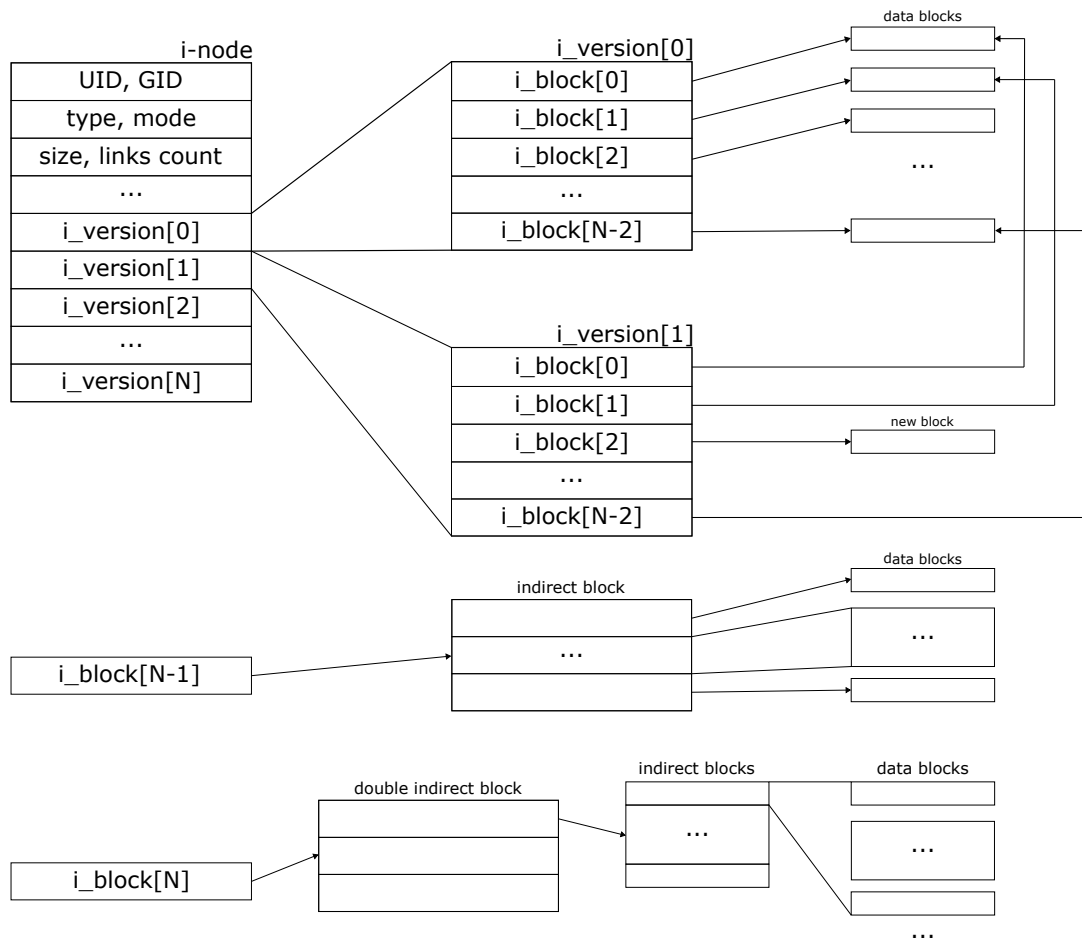


Figure 3.1: Two-layer inode scheme representing a file with two versions. The second version references the same data blocks as the first version, except for the third block. For this block, a new data block is allocated and referenced only by the second version, while the first version continues to reference the original block.

3.1 Versioning Design

This section describes a design approach for implementing file versioning with an emphasis on efficient storage and access. In this approach, the file system maintains three main components: the current file, a version structure, and a separate storage file used to store data. The current file always represents the latest version of the file and is the target of all write operations. When a write operation modifies existing data, the blocks that are about to be overwritten are first copied into the storage file. The version structure then records, for each version, which blocks were overwritten and the corresponding locations in the storage file where the original data is stored. This design allows the current version of the file to be accessed directly, while previous versions can be reconstructed using the information stored in the version structures.

Figure 3.2 illustrates an example of a file that undergoes three consecutive write operations, resulting in three stored versions. Each version is created by opening the file, performing a sequence of write operations, and finally closing the file. This ensures that each stored version represents a consistent state of the file at the moment the write operation is completed.

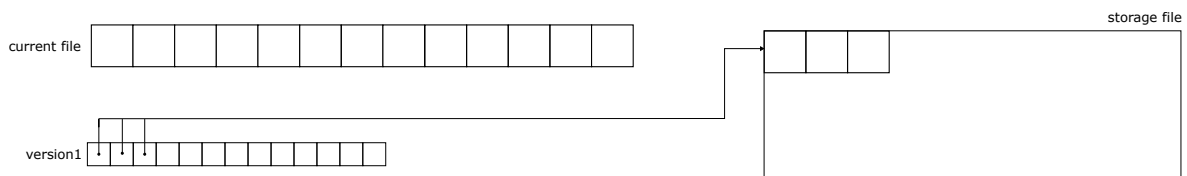
During the first write operation, the first three data blocks of the file are modified. In the second write operation, blocks 3 and 13 are overwritten. Finally, in the third write operation, blocks 3, 4, 5, 8, and 13 are modified. For each write operation, the modified blocks are copied into the version storage, while the version metadata records which blocks were changed and stores pointers to their corresponding locations on disk. Unmodified blocks are referenced from the existing file state, allowing each version to be reconstructed by combining unchanged blocks with the stored modified ones.

To restore a specific version of a file, the corresponding version structure is first examined. Each entry in this structure represents a pointer to a data block. If a block was overwritten in a subsequent version, the original data is stored in the disk file and the entry points to this stored block.

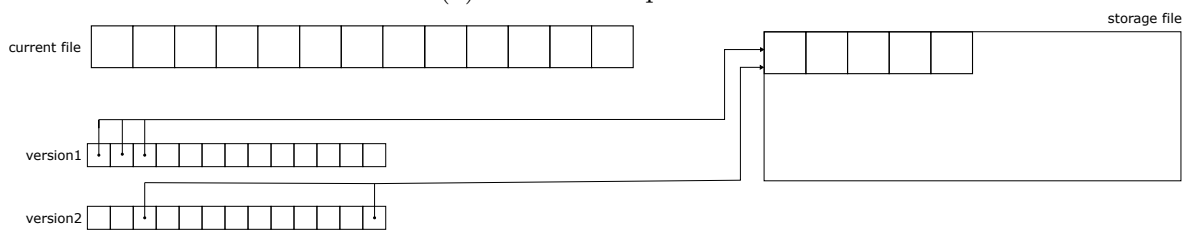
If the block was not modified in the subsequent version (meaning that we did not have to store the block), the recovery process continues by traversing from the selected version towards newer versions. The same check is performed in each subsequent version to determine whether the block was stored. This process is repeated until a version is found in which the block was stored. Once such a version is identified, the corresponding block is selected, as it represents the most recent data for the selected version. In this case, the pointer stored in the version structure is used directly, since each version stores pointers to the blocks that were stored.

If no pointer to a stored block is found in any newer version, the block was never stored, and the block from the current file can be used with certainty.

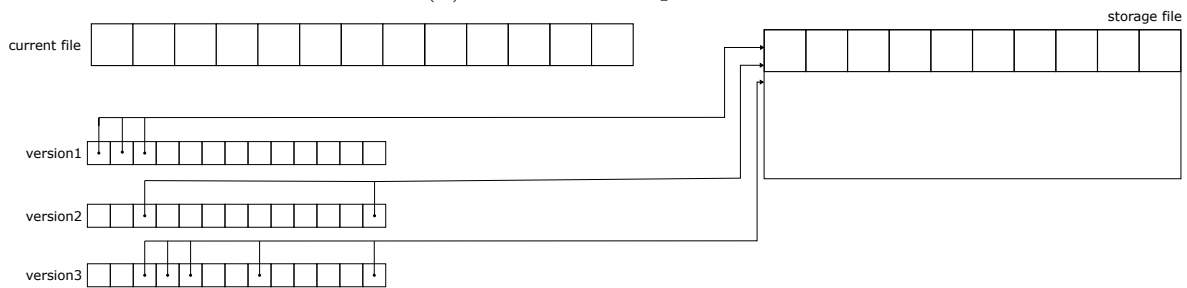
This design provides fast access to the current version of a file, which is accessed



(a) First write operation



(b) Second write operation



(c) Third write operation

Figure 3.2: Structure of a versioned file with multiple write operations

directly as a regular file without additional overhead. Accessing older versions requires a reconstruction process, which introduces additional time overhead. However, compared to the inode-based approach, this solution significantly reduces space usage by avoiding duplication of inode structures and unchanged data blocks. The next section discusses optimization techniques aimed at improving the performance of the reconstruction process and reducing the access time for older file versions.

3.2 Optimization and details

In this section, we discuss optimizations of the algorithm presented earlier in this chapter and provide detailed explanations of each component. The problem with the current design is that it requires traversing versions to retrieve the most recently modified block relative to the selected version. If a block remains unchanged across multiple consecutive versions, resolving it may require traversing many versions, which can negatively affect performance. For this reason, an optimization mechanism is required that traverses the relevant versions once and updates the pointer in the selected version so that it directly references the required version, where the last change to the block occurred. An example of this optimization is shown in Figure 3.3

Figure 3.3 illustrates the structure of the version metadata for a file before and after the optimization process. In the initial state, version entries do not necessarily point directly to the most recent block location. Instead, each entry may reference a next version in order to locate the block, depending on whether it was stored in the current version or in an later one. During a read operation, the system traverses the version chain to locate the correct block. Once the correct block is found, the corresponding version entry is updated along the traversal path. This ensures that all intermediate versions which also rely on this block are updated to directly reference its final resolved location. After the optimization process, all version entries point directly to the most recent location of each block.

In cases where a block was never stored in any subsequent version, the system cannot maintain a direct pointer to a physical disk block, as the current file may still change over time. To handle this, instead of storing a direct pointer to a disk block, the system stores a reference to the version entry past the last one. The actual number of versions is stored in the virtual table file at offset 0, allowing the system to determine whether a referenced version exists. If the reference points to a version entry beyond the existing range, this indicates that the block must be read from the current file. This reference remains valid even as new versions are created, since the version count grows. As a result, the same reference can later resolve to an actual version entry, and traversal continues from that resolved version entry when applicable. If new versions

appear, the version entry reference shortens the actual path that the system needs to travel, making the read faster in later accesses.

As a result, repeated traversal of version file is no longer required, since each version entry pointer is either a direct reference to the disk block or a reference to a newer version entry, effectively shortening the path that the system needs to traverse. This significantly reduces the overhead of future read operations.

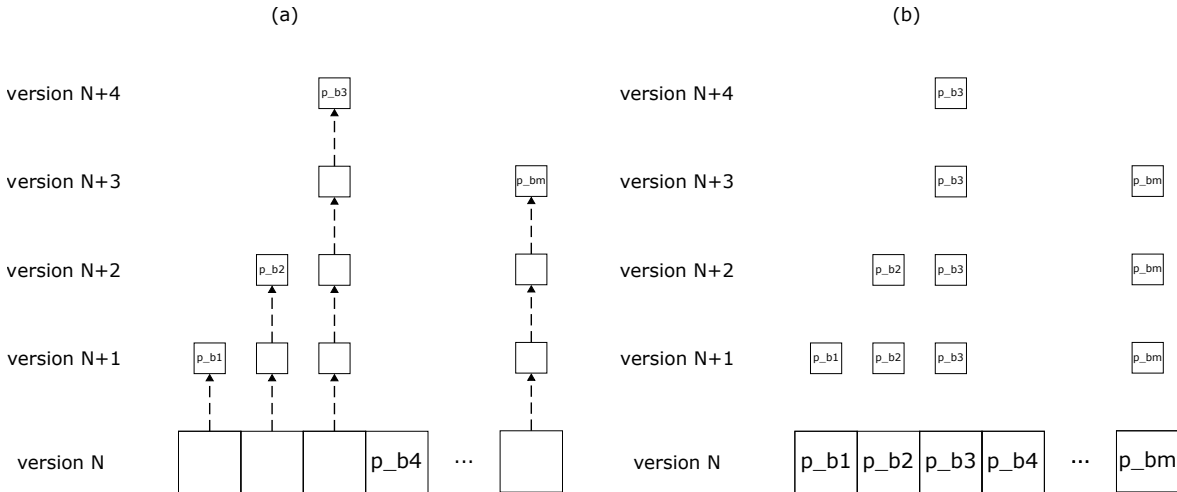


Figure 3.3: Structure of the version entry before and after optimization

Having discussed the optimization, we need a specialized pointer in the version structure, which can refer to either a block on the disk or a subsequent version. From this pointer, it must be clear where it is referencing. Since there are only two possibilities — a block on the disk or a subsequent version — a single bit is sufficient to differentiate between them. The pointer is therefore 64 bits wide: the first bit indicates whether it refers to a disk block or a previous version, while the remaining 63 bits store either the disk address or the version number. Shown in Figure 3.4

Now that the specialized pointer has been introduced, we examine the layout of the version file and how it stores version data. Each version in the file begins with a header containing the version number and the size of the file for that version. Following the header, the version structure contains a sequence of the specialized pointers, one for each block in the version. These pointers indicate either the location of the corresponding disk block or reference a previous version where the block was last modified

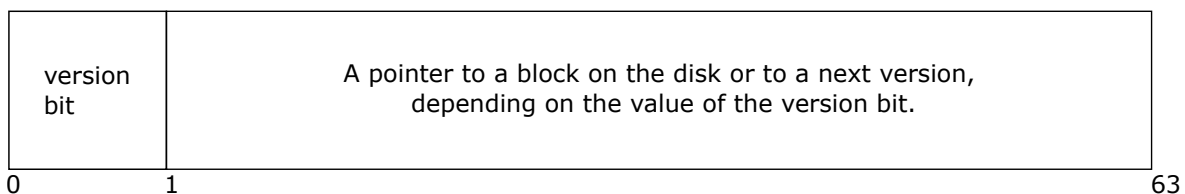


Figure 3.4: Special pointer

(see Figure 3.5). From this version file, it is possible to reconstruct any version of the file by consulting its version structure. However, a problem arises when referencing subsequent versions. To access a specific version, say version X, we must first read the header of the first version to obtain its version number and size. Using the size, we can then skip over its pointers to reach the header of the second version, and repeat this process until reaching version X. This requires scanning the entire version file sequentially, which becomes inefficient for files with many versions. To allow instant access to any version, we introduce an additional structure: the version table. This table contains a mapping from each version number to its corresponding offset in the version file. Because the starting position of a version entry in the version file is already known, storing an explicit version number is unnecessary. Instead, the version number can be replaced with a timestamp representing the moment when the version was created. This provides more useful information to the user, since timestamps can be used when listing available versions, making it easier to identify and select the desired version for recovery. For each file in the mounted directory, two additional files are created within the underlying directory used by the FUSE filesystem: the version file, the version table and the disk file. Using a separate disk file for each file completely eliminates fragmentation caused by shared block storage. When a file is deleted, its corresponding disk file is removed entirely, avoiding the need to free individual blocks and preventing fragmentation of the storage space. For instance, if a file named `myfile` exists in the mounted directory, the underlying directory will contain three corresponding files: `myfile`, `myfile.vf` (the version file), `myfile.vt` (the version table) and `myfile.d` (the disk file). These extensions are relatively common and may collide with user-created filenames. Therefore, the actual implementation uses more complex internal extensions to reduce the probability of such collisions. However, for the sake of readability and clarity, the simplified extensions presented above will be used throughout this thesis. See example in Figure 3.6.

As an optimization, FUSE allows storing additional information about an opened file. In this implementation, a pointer to a custom structure is stored for each opened file. This structure contains relevant data, in particular file descriptors of auxiliary files used for versioning (such as `vf`, `vt`, and the underlying storage file). Since these descriptors are initialized during the open operation, subsequent operations, such as write, do not require repeated opening of metadata files, which reduces overhead.

```
struct my_file_handle {
    int fd_file;
    int fd_vf;
    int fd_vt;
    int fd_disk;
    int is_virtual;
    char path[PATH_MAX];
};
```

```
static int xmp_open(const char *path, struct fuse_file_info *fi)
{
    struct my_file_handle *h = malloc(sizeof(struct my_file_handle));
    // open internal files
    h->fd_vf = fd_vf;
    h->fd_vt = fd_vt;
    //...
    h->is_virtual = 0;
    fi->fh = (uint64_t)h;
    //...
}
```

```
static int xmp_write(const char *path, const char *buf, size_t size,
off_t offset, struct fuse_file_info *fi)
{
    //...
    struct my_file_handle *myfh = (struct my_file_handle *)fi->fh;
    //...
}
```

```
static int xmp_release(const char *path, struct fuse_file_info *fi)
{
    if (myfh->fd_vf)
        close(myfh->fd_vf);
    if (myfh->fd_vt)
        close(myfh->fd_vt);
    //...
    free(myfh);
    //..
}
```

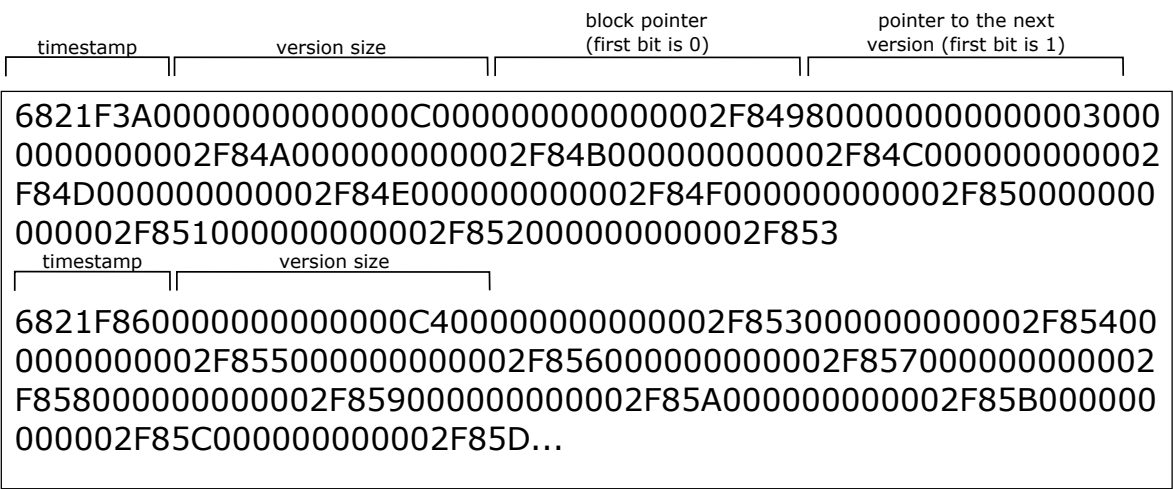



Figure 3.5: Example of a version file.

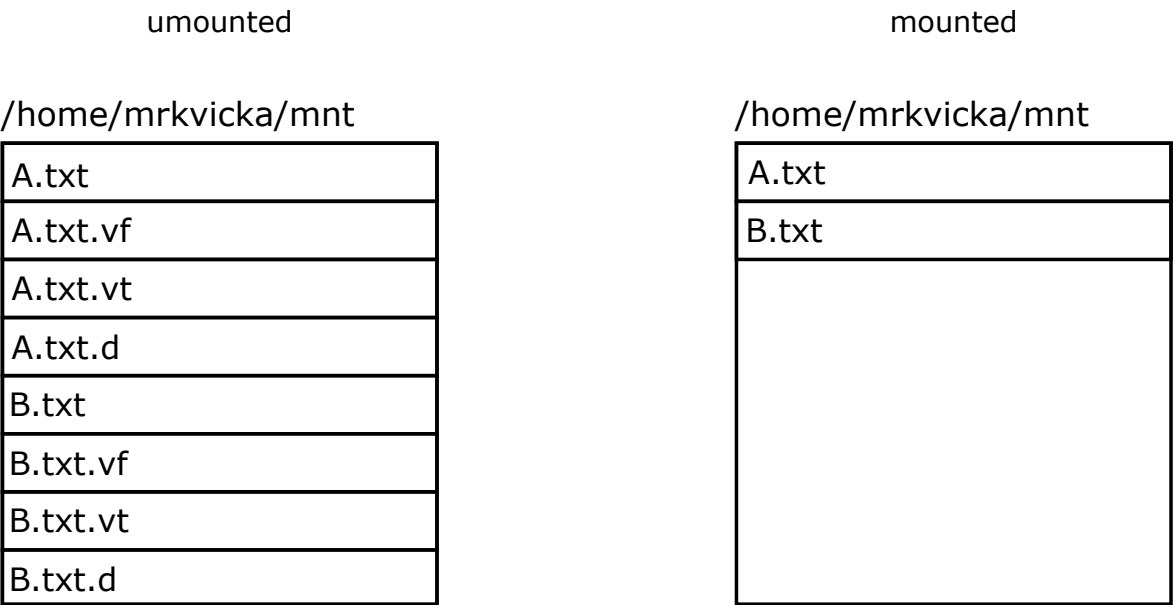


Figure 3.6: Contents of the directory: mounted versus unmounted.

3.3 Version Creation and Internal File Updates

In this section, we describe in detail the conditions under which a new version is created and how new versions are created. The goal is to ensure that each version is created from the file in a consistent state. Let us consider the case where all data are saved. In this approach, a new version would be created for each write operation, and the modified blocks would be recorded. This would allow the file to be restored from any point. Although this approach may seem appealing, consider the following example. Suppose a file is opened and 100 MB of data are written to it. In practice, such a write is performed incrementally, typically in 4 KB blocks. This results in approximately $100 \text{ MB} / 4 \text{ KB} = 25,600$ write operations. Under this approach, this would lead to 25,598 versions, since the initial version of the empty file is not created and the final state corresponds to the current file. This would require storing metadata for each block across all these versions, leading to a significant overhead. From the user's perspective, a single write operation would result in tens of thousands of versions, making it difficult to navigate and identify the desired state.

To avoid this issue, the version must correspond to the state of the file before it is opened for writing by the user. As discussed earlier in the implementation, not all blocks are stored; only the blocks that are overwritten during a single open operation are saved. When a file is opened for writing, a new version entry is created and all existing blocks are marked as unchanged using a special pointer with the version bit set to 1. During a write operation, the affected blocks are determined based on the offset and write size. For each affected block, the corresponding block from the current file is stored on disk and the version entry is updated to reference it. This all applies to a single write operation. Once all affected blocks have been stored and the version entry has been updated to point to the stored blocks, the actual write to the current file can be performed.

This approach introduces several minor issues. Consider the following example. Let us assume a file in the mounted filesystem that starts with "The..." and has no previous versions. We then execute the following command.

```
echo -n "XX" | dd of=mnt/file seek=0 bs=1 conv=notrunc
```

This command writes two bytes, "XX", to the file at offset 0. The argument `bs=1` specifies a block size of 1 byte, meaning that the operation is performed as two separate writes of size 1 byte each. The first write occurs at offset 0, and the second at offset 1. As a result, the first two bytes of the file are overwritten with "XX". From the FUSE perspective, this sequence corresponds to `open("file", ...)`, followed by `write(seek=0, size=1, ...)` then `write(seek=1, size=1, ...)`, and finally `release("file", ...)`. Let us assume that the block size is greater than 1 byte, so both writes affect the same block. Now we

observe the contents of the file during this process. Initially, the file contains "The...". After the first write, the byte at offset 0 is overwritten, and the content becomes "Xhe...". At this point, the original version of the affected block is stored, preserving "The..." as the previous state. However, the second write introduces an important issue. It targets the same block, and the file content becomes "XXe...". In this step, the affected block from the current file is saved, which now contains "Xhe...". As a result, the original correct block corresponding to the version "The..." is overwritten by the intermediate state of the block from the current file, rather than preserving its original content. This leads to two problems. The first, as observed above, is that the version is constructed from blocks of the current file that are in an inconsistent state. The second problem is that inconsistent blocks are stored on disk after each write. When multiple writes target the same block, intermediate and inconsistent versions of that block may be saved repeatedly. However, these stored blocks are never actually referenced, since each new write overwrites the corresponding pointer in the version entry. As a result, the system may accumulate a large number of unused blocks on disk that are not reachable from any version. What is actually required is to store the original block from the current file only once and maintain a single valid pointer to it. This is the one valid pointer, and we must never overwrite it. To fix this issue, we first need to check, before storing any block, whether it has already been stored. This can be determined from the pointer: if it references the next version (i.e., the version bit is set to 1), then the block has not yet been stored. If the pointer already references a block on disk, it must not be overwritten. With this minor modification, the system correctly preserves the blocks required to restore a consistent version, and unnecessary block storage is avoided.

3.4 Methods of Accessing File Versions

In this section, we discuss different approaches to accessing file versions. This section does not address the reconstruction of versions from the metadata file; instead, it focuses on how versions are accessed and the mechanisms used to provide this functionality.

There are two main approaches that can be considered for accessing file versions:

- Separate program – a dedicated application responsible solely for accessing and retrieving file versions.
- FUSE-based approach – integration of version access directly within the FUSE filesystem implementation.

The first approach is the simplest to implement. A dedicated program would identify the target file and the requested version, reconstruct the corresponding data, and

output it, for example, to the standard output. The advantage of this approach is its simplicity of implementation. The program only needs to obtain the path to the file, access the internal data structures that store version information, apply the reconstruction method described earlier, and output the result to the standard output. Additionally, the program can provide parameters or options that allow the user to either list all available versions or select a specific version to retrieve. This raises the following question: what problem does this approach introduce? As we mentioned in this work, we want the mounted filesystem to use the directory where it was mounted for security reasons. However, our core idea was to ensure that the user does not see internal files used to store version information. Once the FUSE filesystem is mounted, there is no way to access these internal files and restore versions. A possible solution is to start the version-reading program before mounting the FUSE filesystem, open the directory where the filesystem will be mounted, and retain the file descriptor in order to access the internal files. However, this approach would be very inconvenient. Although this approach is acceptable when using a separate directory to store internal data and maintain access to it, in this work it would contradict one of our main design principles.

The second approach is to implement version management directly within the FUSE filesystem. At first glance, this may appear confusing, as it is not possible to introduce custom operations into the VFS. This restriction exists because the Virtual File System layer in the Linux kernel defines a fixed interface between user space and filesystem implementations, where all communication is translated into a small, predefined set of operations such as lookup, getattr, open, and read, among others. Since these operations are hardcoded in the kernel's filesystem API, there is no mechanism for a filesystem like FUSE to introduce new syscall-like operations, as every user request must be mapped onto one of the existing VFS-defined entry points. However, version access can be implemented using the existing filesystem operations, and this can be done in a rather elegant way. We need to support two main functionalities: listing the available versions of a file and selecting and reading a specific version. We aim to trigger an action that results in the display of a list of available versions. So what is this similar to? This behavior is very similar to reading a directory, where its contents are listed. Therefore, we can model the versions as a directory that contains all versions of the corresponding file. The following idea, which naturally follows from this model, is to treat reading a version of a file as opening and reading a file from this directory. At this point, we create a directory for each file and, within it, a separate file for each version of that file. It is important to note that these files are empty. Their purpose is purely to act as triggers for version access. In reality, when such a file is accessed, the corresponding version is reconstructed from metadata stored in the internal files. The empty file itself does not contain any data; it only serves as an interface element

that initiates the retrieval process. There are two issues with this approach. It violates UNIX expectations by decoupling file size from readable content, since a zero-sized file may still produce data through `read()`. It also leads to misleading metadata, as many tools and system components rely on file metadata before reading, causing a zero-sized file to appear empty despite dynamically generating content on access. This is where the hard part of this approach begins, in contrast to the previous one. To make this model even better, we have to refrain from creating something new and instead emulate their presence within the existing structure. Here we introduce the concept of virtual files. The idea is to emulate the existence of a directory for each file, containing versions within it. The main difficulty of this approach lies in handling these virtual entries for every file operation. The closer virtual files behave like real files, the better compatibility they achieve with standard tools and expected filesystem behavior. For example, in the `getattr()` operation, the file size and the version creation time can be set based on the values stored in `.vf` file associated with each version.

Chapter 4

Testing

The goal of this chapter is to test the implemented system and evaluate its behavior under real workloads. The system is benchmarked to measure its performance and correctness. The results are compared with existing approaches in order to assess the advantages and limitations of the design. Based on these results, we draw conclusions about the system. The chapter is divided into two sections: testing write operations and testing read operations. Each section focuses on its own goal, as indicated by the title. The write section evaluates the performance and behavior of the system under write workloads, while the read section focuses on the correctness and efficiency of version retrieval.

4.1 Testing write

In this section we describe the write testing in detail. Since this filesystem is designed to handle sensitive files such as documents, we test it using the following file sizes: 100 KB, 100 MB, 500 MB, and 3 GB. Small file sizes are used to evaluate the behavior of small writes within the filesystem, while larger sizes are used to test full-file operations. The 3 GB case is included to observe the system's behavior under very large file workloads. The testing was performed on the same machine using the `fio` [4] tool. Each test was executed multiple times, and the results were averaged to obtain more stable measurements. With the provided table, we analyze the results and attempt to explain the observed differences between the tested systems. The results of the write testing are shown in Table 4.1.

- **ext4** – traditional Linux filesystem
- **pass** – FUSE passthrough filesystem with no modifications
- **versioned** – implementation of the versioned FUSE filesystem

- **versioned (empty file)** – versioned filesystem tested on a fresh file without prior data, used to measure the overhead of versioning in the absence of existing data

Note: The reported slowdown is calculated as the relative performance difference between the versioned filesystem and the passthrough (**pass**).

File size	ext4	pass	versioned	versioned (ef)	slowdown (%)
100 KB	<1 ms	4.4 ms	3.7 ms	3.9 ms	-15.9%
100 MB	63 ms	816 ms	1647 ms	1074 ms	101.9%
500 MB	255 ms	4331 ms	8225 ms	5158 ms	89.9%
3 GB	1298 ms	21088 ms	40407 ms	30098 ms	91.6%

Table 4.1: Write performance comparison and slowdown relative to passthrough

Across all tested file sizes, the traditional filesystem (ext4) consistently outperforms both the passthrough and versioned filesystem implementations. This is expected, as ext4 operates in-kernel and avoids the overhead of user-space FUSE processing and additional metadata management.

The passthrough implementation introduces a consistent performance overhead compared to ext4 across all workloads due to FUSE indirection and user-space execution costs. However, it remains significantly faster than the versioned filesystem in most scenarios.

The versioned filesystem shows the highest latency overall, and this overhead increases with file size as more metadata must be maintained for version tracking. However, in the case of empty files (versioned (ef) case), performance improves significantly since there is no historical data to manage. In this scenario, the versioned filesystem approaches the performance of passthrough but remains slightly slower due to the unavoidable overhead of metadata handling and version management logic.

Overall, the results confirm that while versioning introduces substantial overhead in general workloads, its cost is partially reduced when there is no meaningful version history, though it still does not match the efficiency of a non-versioned passthrough implementation.

4.2 Testing read

For read performance testing, the most extreme case is examined for a relatively representative file size. As discussed previously, the main drawback of reading older versions is the need to traverse the version file in order to locate the correct block corresponding to a given version.

In this test, a 100 MB file is used. To simulate a worst-case scenario, the first block of the file is overwritten 1000 times, resulting in 999 stored versions. For all blocks except the first one, the system will have to traverse through the entire version file and eventually read the blocks from the current file. After preparing the file in this state, different versions are read multiple times, and the read time is measured. The final result is obtained as the average of these measurements.

Version	Read time
Current version	639 ms
999th version	1143 ms
900th version	2838 ms
500th version	12344 ms
1st version	23288 ms

Table 4.2: Read performance of different file versions

It should also be noted that, on the passthrough FUSE filesystem, the read time for the same 100 MB file was measured at 674 ms, which is approximately the same as in the versioned case for the current file. This is expected, since reading the current file in the versioned filesystem is effectively equivalent to a direct file read through FUSE. These results are consistent with the expected behavior of the system. The most important observation is that the current version is read in the shortest time. This confirms that the system achieves its intended goal of prioritizing access to the most recent version of a file, which is also the version that is most frequently accessed by the user.

Chapter 5

Alternative approach

In this chapter we present an alternative approach to storing file versions in the filesystem. The main idea is to modify the way version data is stored in order to improve certain aspects of the previous implementation. This comes at the cost of some of the properties that were beneficial in the original approach, meaning that the new design represents a trade-off between different system goals. In this approach, the main goal is to make all versions equally accessible, meaning that accessing both the latest and older versions has similar cost and is not prioritized in the retrieval process. However, this comes at the expense of slower access to the latest version of the file compared to the previous implementation.

5.1 Implementation details

This implementation is largely similar to the previous one and uses the same files for storing version information. As mentioned previously, the goal of this implementation is to make all versions equally accessible. This means that reconstructing a version does not require traversing the version file. Instead, each version directly stores pointers to the corresponding blocks on disk. To make this approach work, several aspects must be handled differently from the previous implementation. In the previous design, only the blocks that were about to be modified were stored on disk. As a result, a version entry contained pointers only to the blocks that had changed. In this implementation, however, each version must contain pointers to all blocks in order to allow direct reconstruction of the version without traversing subsequent entries. To achieve this, instead of storing blocks before they are overwritten, the current state of the file is stored after the write operation is completed. In more technical terms, the file is first modified and only afterwards its contents are saved, unlike in the previous implementation where blocks were stored before the file was changed. However, this approach introduces another issue. Since the file is first modified and only afterwards stored as a version, the

current file and the latest created version become identical. As a result, storing both separately provides no benefit while significantly increasing storage usage. For example, consider a file with a size of 100 GB. In this case, the system would store 100 GB as the current file and another 100 GB as the latest version on disk (this is true only if we start with an empty file or overwrite the entire 100 GB of data, because we store on disk only the modified blocks). This effectively doubles the required storage space for the same data. This leads to the conclusion that maintaining a separate current file is unnecessary in this design. Instead, the latest version itself can be treated as the current file and accessed directly. Now this means that the current file is not read as a regular file; instead, the pointers stored in the version entry must be resolved. This makes reading the current file slower, which is an important drawback, since the user typically accesses the current version in most cases and uses older versions only for recovery or in case of errors.

Let us go over the details of how a version is stored. As in the previous implementation, there are two types of blocks: changed and unchanged, with different handling depending on their type. If a block is changed in the current write, it is written to disk and the pointer to it is updated in the version entry. If a block is not changed, the pointer is simply taken from the previous version. The previous version always contains valid pointers to disk blocks. We start from an empty file, so each written block is initially new, stored on disk, and referenced from the version entry. From this point onward, each pointer is either carried over to the next version or replaced with a new valid pointer if the corresponding block is modified. It is also important to note that, as in the previous implementation, care must be taken when the file is opened for writing and multiple write operations occur. In the previous approach, the goal is to store each changed block only once, since data is taken from the current file and the corresponding block in the current file is then overwritten. In this implementation, however, since we store the current file or the last version representing the current file, the last write becomes the most relevant, and only the blocks stored in that final write overwrite all previous intermediate ones.

The main distinction between the two approaches is that in the previous implementation, the goal is to preserve the contents of the file as they were before the file was opened for writing. In this approach, however, we store data as it is being written, meaning that the last written state is the one that is stored, and it overwrites all intermediate states. For the same reasons, the size of the version is stored when the file is closed, since the actual size of the version is only known after all modifications are completed. In contrast, in the previous implementation the size is stored when the file is opened, because it represents the state of the file before any modifications occur.

The process of reading a version in this approach is significantly simpler than in the previous implementation. For each block, the system only needs to read the cor-

responding pointer from the version entry and then retrieve the block from disk at that address. Because all versions store direct pointers to disk blocks, the reading time for both the latest and older versions is the same. This approach is much easier to implement and solves the problem of reading older versions. However, we will examine its cost in the next testing section.

5.2 Testing the new approach

After implementing this approach, it is tested in the same manner as the previous implementation in order to ensure a fair comparison. The primary objective of this evaluation is to verify that reading both the newest and the oldest versions is performed in a similar amount of time, as intended by the design.

Proceeding with the reading tests, the results are shown in Table 5.1. As can be seen, the read times of the different versions are approximately the same. This behaviour is expected in this implementation, since all versions are resolved directly through stored block pointers without the need for traversing a version history or performing any reconstruction of file contents.

Version	Read time
999th version	1840 ms
900th version	1907 ms
500th version	2073 ms
1st version	1872 ms

Table 5.1: Read performance of different file versions

Compared to the previous implementation, this approach provides approximately uniform read times for both older and newer versions of the file. In the previous design, access to newer versions was optimized, while older versions required additional traversal through version metadata, resulting in progressively higher read times as the version history grew. In contrast, the current implementation removes this distinction by storing direct pointers to all relevant blocks, ensuring that each version can be reconstructed in a similar manner.

The small variation between individual measurements is mainly caused by caching effects and minor differences in disk access patterns during execution.

Another consequence of this design is related to the physical layout of data blocks within the storage file. When a file is modified, newly written blocks are appended to the end of the disk file, while unchanged blocks remain stored at their original locations near the beginning. As a result, partially modified files no longer benefit from a fully sequential layout. During sequential reads, the system may need to alternate between

blocks stored in different regions of the disk, which introduces additional seek overhead. This can negatively impact read performance compared to a traditional contiguous file layout, particularly as the number of modifications increases and the distance between block locations grows.

Overall, the results confirm that this implementation successfully decouples version access time from version age, achieving uniform read performance across all stored versions.

Future Work

The presented design of the versioned file system is flexible and can be extended with relatively small modifications to the existing implementation. One of the most natural extensions would be the introduction of more advanced version management mechanisms. This includes the ability to delete specific versions or to implement policies for limiting the number of stored versions. At present, all versions are preserved, which simplifies the implementation but may lead to unnecessary growth of version metadata over time.

Another important direction for future work is improving the efficiency of version handling by reducing the number of system calls required during file operations. In the current implementation, certain actions related to version creation and management introduce additional system-level interactions that are not strictly necessary for correctness. These operations are a consequence of how version entries are constructed and updated.

Overall, future improvements should focus on refining version management and reducing unnecessary system-level overhead while preserving the core functionality of file versioning and recovery.

Conclusion

This thesis presented the design, implementation, and evaluation of a versioned file system built using FUSE. The primary goal of the system was to provide file versioning capabilities that enable recovery of previous file states while maintaining acceptable performance during regular file operations. Two different implementation approaches were developed and analysed in order to explore the trade-offs between access speed, storage structure, and versioning consistency.

The first implementation prioritises fast access to the current version of a file. In this design, modifications are applied directly to the current file, while previous states are preserved in a way that allows reconstruction of older versions when needed. This approach results in efficient access to the most frequently used data, which is the current version of the file. However, the cost of this design is that accessing older versions requires additional traversal and reconstruction steps, leading to increased read times for historical data.

The second implementation takes an alternative approach by ensuring that all versions are accessible in approximately the same amount of time. This is achieved by storing direct pointers to all relevant blocks, allowing both old and new versions to be reconstructed in a uniform manner. As a result, the difference between accessing the newest and oldest versions is significantly reduced. However, this design introduces additional overhead during write operations and results in less efficient access to the current version, which is expected to be the most frequently accessed state in typical usage scenarios.

Based on the evaluation results, the two implementations demonstrate different strengths depending on the intended use case. The first approach is better suited for environments where the primary goal is efficient access to the latest version of a file, and older versions are only occasionally used for recovery purposes. In contrast, the second approach is more appropriate in scenarios where uniform access to all versions is required, and the system must treat historical and current states equally.

The testing results further confirm that both designs successfully achieve their intended objectives, but with different performance trade-offs. The first implementation optimises for common-case performance at the cost of historical access efficiency, while the second implementation balances version access times but introduces additional

overhead in write operations and current file access.

Overall, this work demonstrates that the design of a versioned file system is strongly dependent on the expected usage pattern. Optimising for one aspect of performance inevitably affects another, and the choice of implementation should therefore be guided by whether the system prioritises recovery efficiency, normal operation speed, or balanced version access.

Bibliography

- [1] The BTRFS file system. <https://btrfs.readthedocs.io/en/latest/Introduction.html>. Accessed: 2026-01-04.
- [2] Overview of ZFS snapshots. <https://docs.oracle.com/cd/E19253-01/819-5461/gbchx/index.html>. Accessed: 2026-01-04.
- [3] The ZFS file system. https://openzfs.org/wiki/Main_Page. Accessed: 2026-01-04.
- [4] Jens Axboe. fio - flexible i/o tester. https://fio.readthedocs.io/en/latest/fio_doc.html. Accessed: 2026-05-02.
- [5] The Linux Kernel Documentation. The ext4 file system. <https://www.kernel.org/doc/html/v5.0/admin-guide/ext4.html>. Accessed: 2026-01-04.
- [6] The Linux Kernel Documentation. Fuse (filesystem in userspace). <https://www.kernel.org/doc/html/next/filesystems/fuse.html>. Accessed: 2026-01-04.
- [7] The Linux Kernel Documentation. The XFS file system. <https://docs.kernel.org/admin-guide/xfs.html>. Accessed: 2026-01-04.
- [8] OpenAI. Chatgpt (free version). <https://chatgpt.com>, 2026. Used for language proofreading and assistance in identifying relevant literature sources; not used as a primary source of information.

User Manual

This section describes how to build and run the implemented filesystem.

Build

The filesystem can be built using the following GCC command:

```
gcc -fdiagnostics-color=always -Wall \  
vfile_handler.c vtable_handler.c fs_helper.c myfs.c \  
'pkg-config fuse3 --cflags --libs' \  
-o myfs
```

This produces the executable named `myfs`.

Usage

The filesystem can be mounted using:

```
./myfs mnt/
```

The mount point directory (`mnt/`) is optional and can be replaced with any existing directory path.